МИНОБРНАУКИ РОССИИ
САНКТ–ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЁТ

по лабораторной работе №1
по дисциплине «Машинное обучение»
Тема: «Классификация»

Студент гр. 2387 _____ Недовизий А.И.

Преподаватель _____ Турсуков Н.О.

Санкт–Петербург
2026

Оглавление

Цель работы 3
Задачи работы 3
Теория 3
Загрузка и первичный анализ данных 5
Корреляционный анализ 5
Визуализация значимых признаков 6
Очистка данных 8
Генерация данных для балансировки классов 8
Обучение моделей и подбор параметров 9
Оценка качества моделей 9
Вывод 14
Приложение А 15
Исходный код программы 15

Цель работы

Получение и закрепление навыков предобработки данных и применения методов машинного обучения для решения задач классификации.

Задачи работы

1. Провести предварительную обработку данных: визуализировать значимые признаки, выполнить очистку данных (удаление пропусков, дубликатов, нормализация), построить матрицу корреляций.
2. Написать скрипт генерации данных для приведения набора к сбалансированному виду без использования готовых методов.
3. Обучить модели K-ближайших соседей (KNN), машины опорных векторов (SVM) и случайного леса с подбором гиперпараметров.
4. Оценить качество моделей с помощью метрик precision, recall, f1-score, ROC-AUC, построить матрицы ошибок, визуализировать важность признаков для случайного леса.

Теория

Задача классификации – это задача машинного обучения с учителем, в которой требуется отнести объект к одному из заранее известных классов на основе его признаков. В данной работе решается задача многоклассовой классификации с четырьмя классами и сильным дисбалансом.

Метод K-ближайших соседей (KNN) – алгоритм, который относит объект к тому классу, к которому принадлежит большинство из K его ближайших соседей в пространстве признаков. Основные гиперпараметры: число соседей K, метрика расстояния, вес соседей.

Машина опорных векторов (SVM) – алгоритм, строящий разделяющую гиперплоскость, максимизирующую зазор между классами. Для нелинейных задач

используется ядровой трюк (например, RBF-ядро). Основные гиперпараметры: параметр регуляризации C, тип ядра, коэффициент gamma.

Случайный лес (Random Forest) – ансамблевый метод, состоящий из множества решающих деревьев, каждое из которых обучается на случайной подвыборке данных и случайном подмножестве признаков. Итоговое решение принимается голосованием. Основные гиперпараметры: число деревьев, максимальная глубина, минимальное число объектов для разбиения.

Дисбаланс классов – ситуация, когда один класс представлен значительно большим числом объектов, чем другие. Это приводит к смещению модели в сторону мажоритарного класса. Для борьбы с дисбалансом применяется метод SMOTE (Synthetic Minority Over-sampling Technique), который генерирует синтетические объекты миноритарных классов на основе интерполяции между ближайшими соседями.

Метрики качества:

Accuracy – отношение числа верных предсказаний модели к общему количеству прогнозов.

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision – доля верно предсказанных объектов среди всех, отнесённых моделью к данному классу.

$$precision = \frac{TP}{TP + FP}$$

Recall – доля верно предсказанных объектов среди всех реальных объектов данного класса.

$$recall = \frac{TP}{TP + FN}$$

F1-score – гармоническое среднее precision и recall.

$$recall = 2\,\frac{recall * precision}{recall + precision}$$

ROC-AUC – площадь под ROC-кривой; для многоклассовой задачи усредняется по схеме one-vs-rest.

Загрузка и первичный анализ данных

В качестве набора данных выбран датасет Perfume Type Classification (Kaggle), содержащий 70 000 записей и 11 столбцов. Таргетная переменная – perfume_category с четырьмя классами: Eau de Cologne, Eau de Toilette, Eau de Parfum, Perfume Extract.

На рисунке 1 виден очевидный дисбаланс целевого класса.

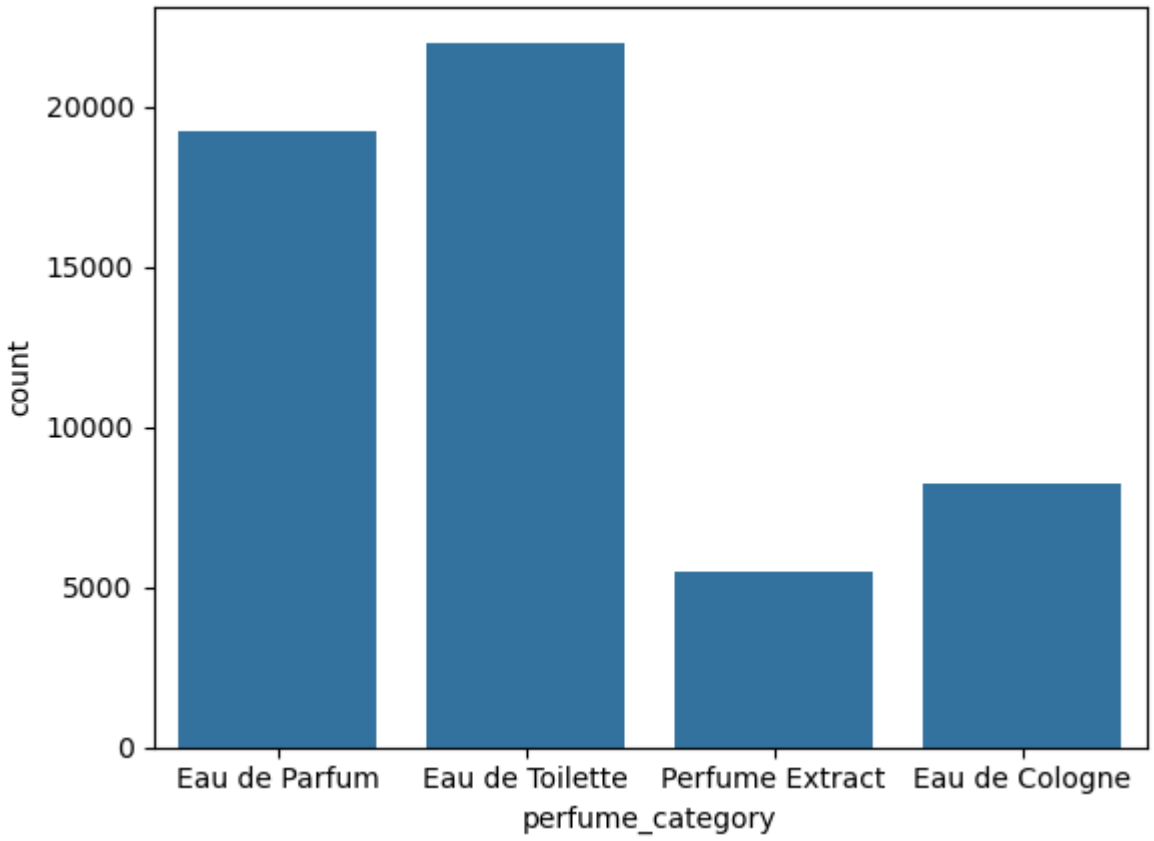


Рисунок 1 – Распределения таргетных классов.

Корреляционный анализ

Построена матрица корреляций числовых признаков (рисунок 2).

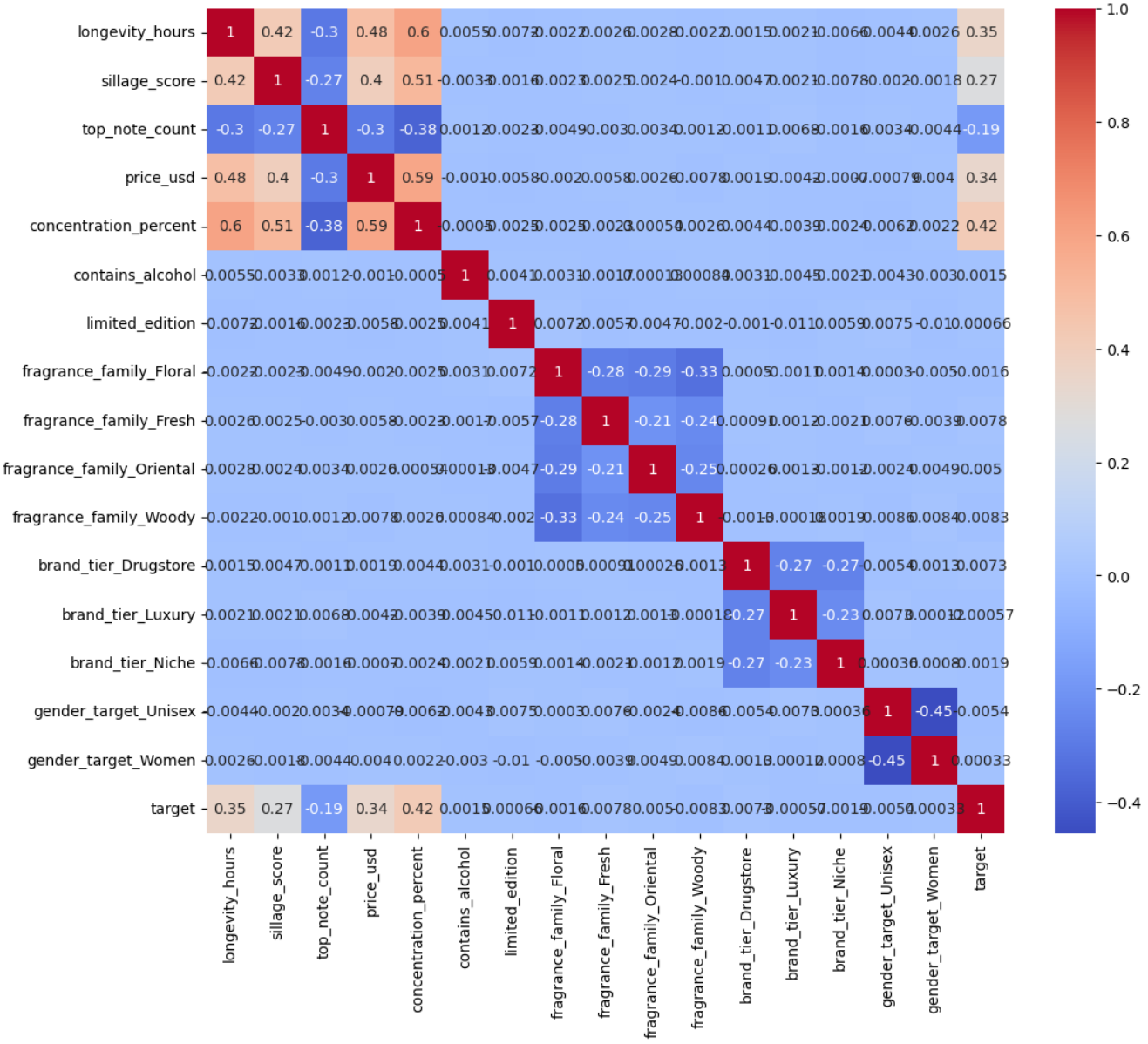


Рисунок 2 – Матрица корреляций числовых признаков

Визуализация значимых признаков

Построены диаграммы распределения наиболее важных признаков. Для числовых признаков построены kdeplot и violinplot, что может являться несколько избыточным поскольку на violinplot уже есть kde распределение, однако я считаю, что kdeplot более нагляден поскольку все распределения на одной оси.

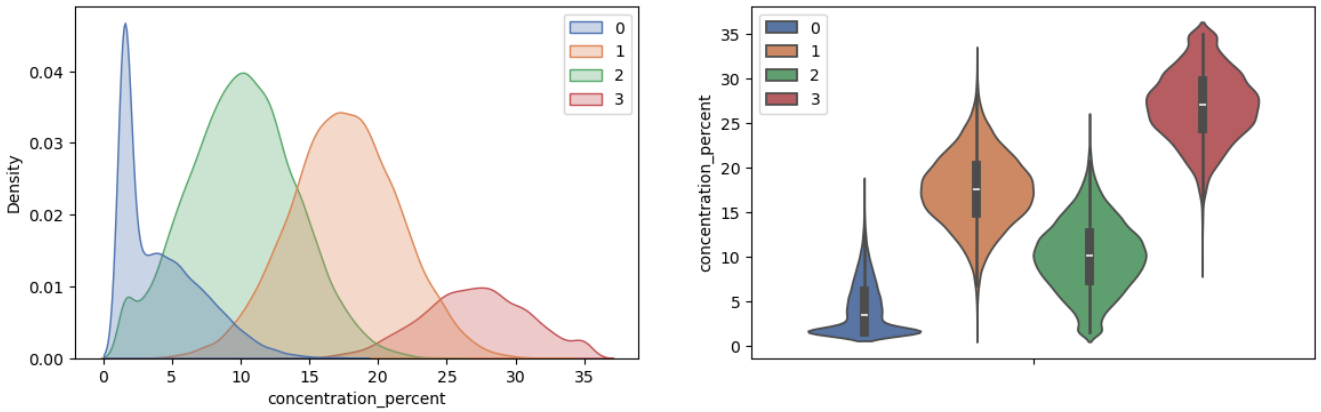


Рисунок 3 – Диаграммы для признака concentration_percent

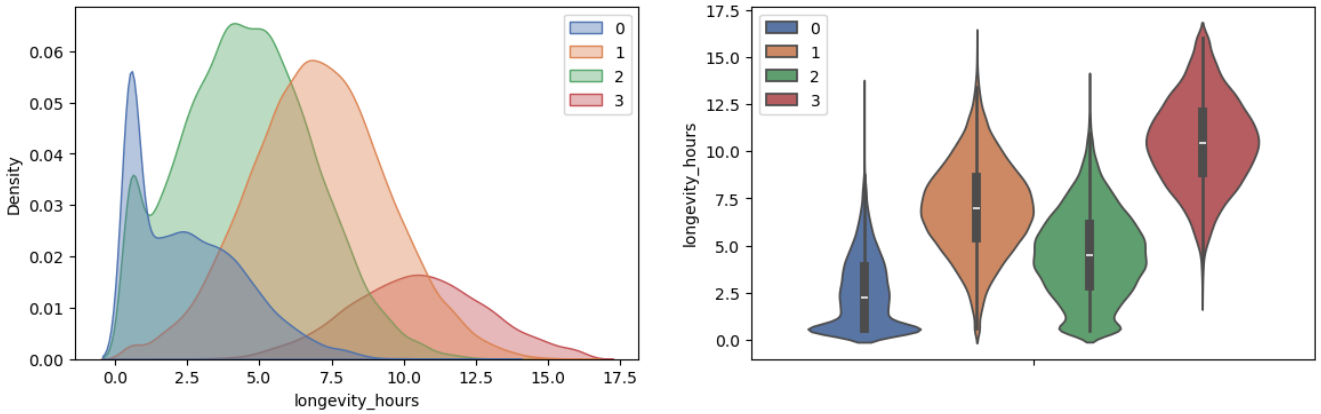


Рисунок 4 – Диаграммы для признака longetivity_hours

Для категориальных данных были дополнительно построены scatter plot и histplot для лучшей наглядности распределения численных данных по таргетным классам.

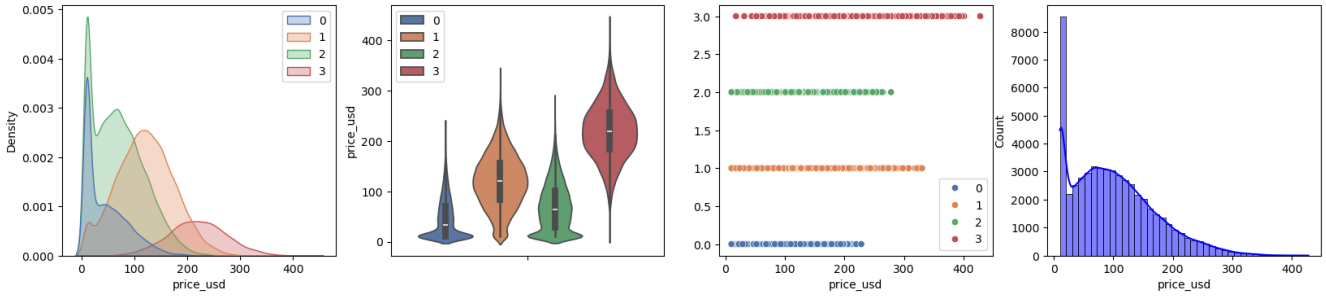


Рисунок 5 – Диаграммы для признака price_usd

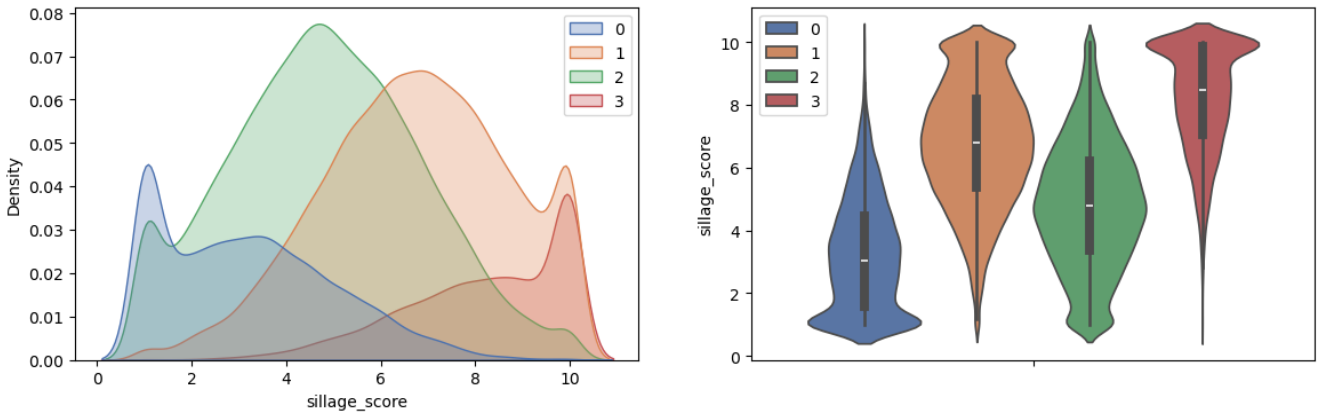


Рисунок 6 – Диаграммы для признака sillage_score

Очистка данных

На этапе очистки выполнены следующие действия: Удалены дубликаты (15к строчек), пропуски заполнены медианой для численных признаков, а для категориальных введен новый класс “Unknown”.

Генерация данных для балансировки классов

Для устранения дисбаланса написан самописный скрипт генерации данных на основе метода SMOTE. Алгоритм работает следующим образом: для каждого миноритарного класса находятся K ближайших соседей каждого объекта, затем между объектом и случайно выбранным соседом генерируется новая точка путём выбора случайной точки между бейз семплом и соседом. Категориальные признаки наследуются от бейз семлпа. Генерация повторяется до тех пор, пока не будет сгенерировано len(minor_class) * oversampling_rate, то есть количество данных зависит от коэффициента, а не до полного совпадения числа семплов. Сделал это для того, чтобы можно делать примерный баланс, а не идеальный, поскольку имеющегося количества семплов на класс хватит чтобы понять признаки, но баланс такой чтобы не переобучаться под конкретный класс.

```
исходный баланс в трейне:
[ 6607 15403 17611  4379]

 баланс классов в трейне после генерации:
[15856 15403 17611 16640]
```

Рисунок 7 – Баланс классов до и после генерации синтетических семплов

Обучение моделей и подбор параметров

Числовые признаки масштабированы с помощью StandardScaler, категориальные – закодированы через OneHotEncoder. Подбор гиперпараметров выполнен с помощью GridSearchCV (3-кратная кросс-валидация, метрика – F1-macro).

```
Запуск GridSearchCV для KNN...
Fitting 3 folds for each of 6 candidates, totalling 18 fits
Лучшие параметры для KNN: {'classifier__n_neighbors': 7, 'classifier__weights': 'distance'}
Лучший F1-Macro на кросс-валидации: 0.7898

Запуск GridSearchCV для SVM...
Fitting 3 folds for each of 3 candidates, totalling 9 fits
Лучшие параметры для SVM: {'classifier__C': 1, 'classifier__kernel': 'rbf'}
Лучший F1-Macro на кросс-валидации: 0.8438

Запуск GridSearchCV для Random Forest...
Fitting 3 folds for each of 9 candidates, totalling 27 fits
Лучшие параметры для Random Forest: {'classifier__max_depth': 20, 'classifier__n_estimators': 800}
Лучший F1-Macro на кросс-валидации: 0.8473
```

Рисунок 8 – Результаты нахождения лучших параметров с GridSearch

Оценка качества моделей

Для каждой модели выведен отчёт о классификации и построена матрица ошибок, они приведены на рисунках 9 – 11.

	precision	recall	f1-score	support
Eau de Cologne	0.65	0.79	0.71	1651
Eau de Parfum	0.84	0.80	0.82	3851
Eau de Toilette	0.79	0.73	0.76	4403
Perfume Extract	0.81	0.93	0.86	1095
accuracy			0.78	11000
macro avg	0.77	0.81	0.79	11000
weighted avg	0.79	0.78	0.78	11000

ROC-AUC Score (One-vs-Rest, Macro): 0.9357

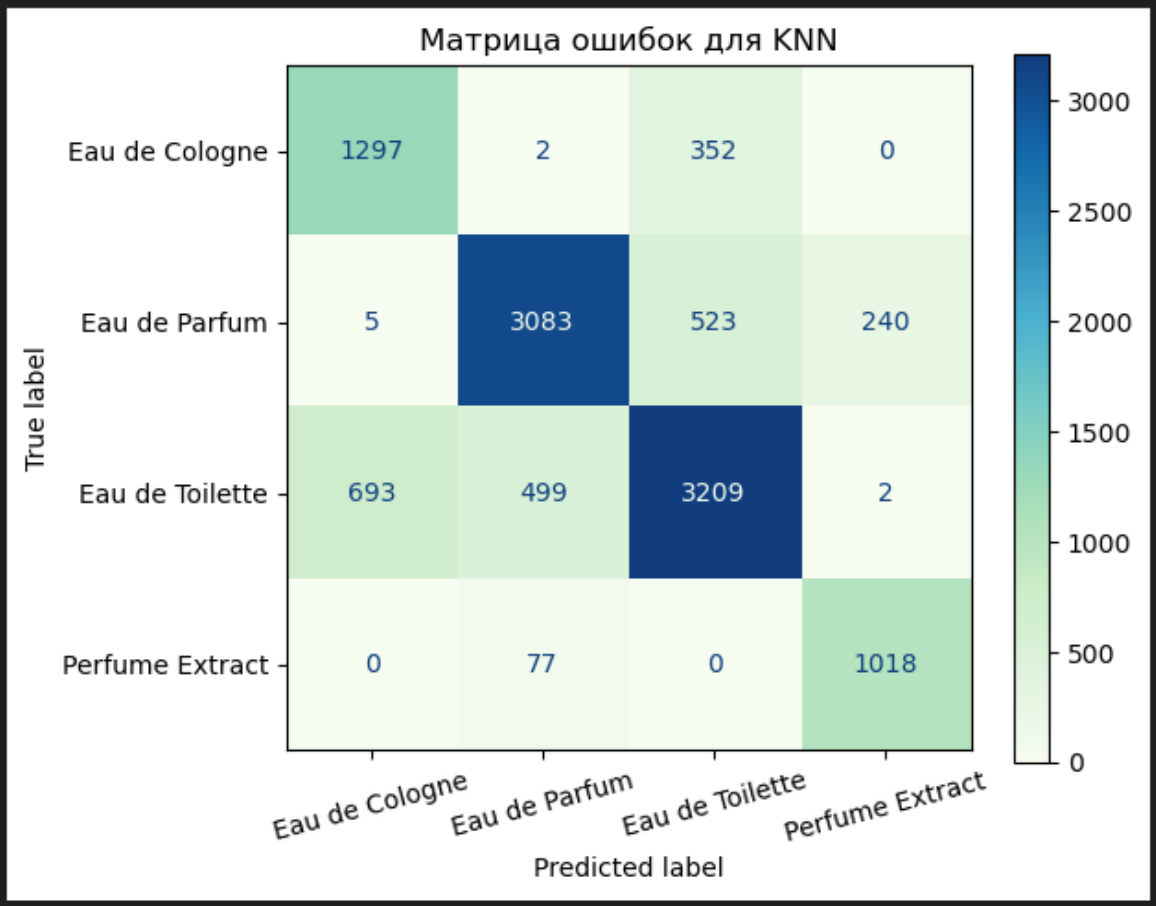


Рисунок 9 – Classification report и confusion matrix модели KNN

```
                  precision    recall  f1-score   support

   Eau de Cologne       0.74      0.83      0.78      1651
    Eau de Parfum       0.88      0.85      0.87      3851
Eau de Toilette         0.83      0.80      0.81      4403
Perfume Extract         0.88      0.95      0.91      1095

         accuracy                           0.84     11000
        macro avg       0.83      0.86      0.84     11000
     weighted avg       0.84      0.84      0.84     11000

ROC-AUC Score (One-vs-Rest, Macro): 0.9652
```

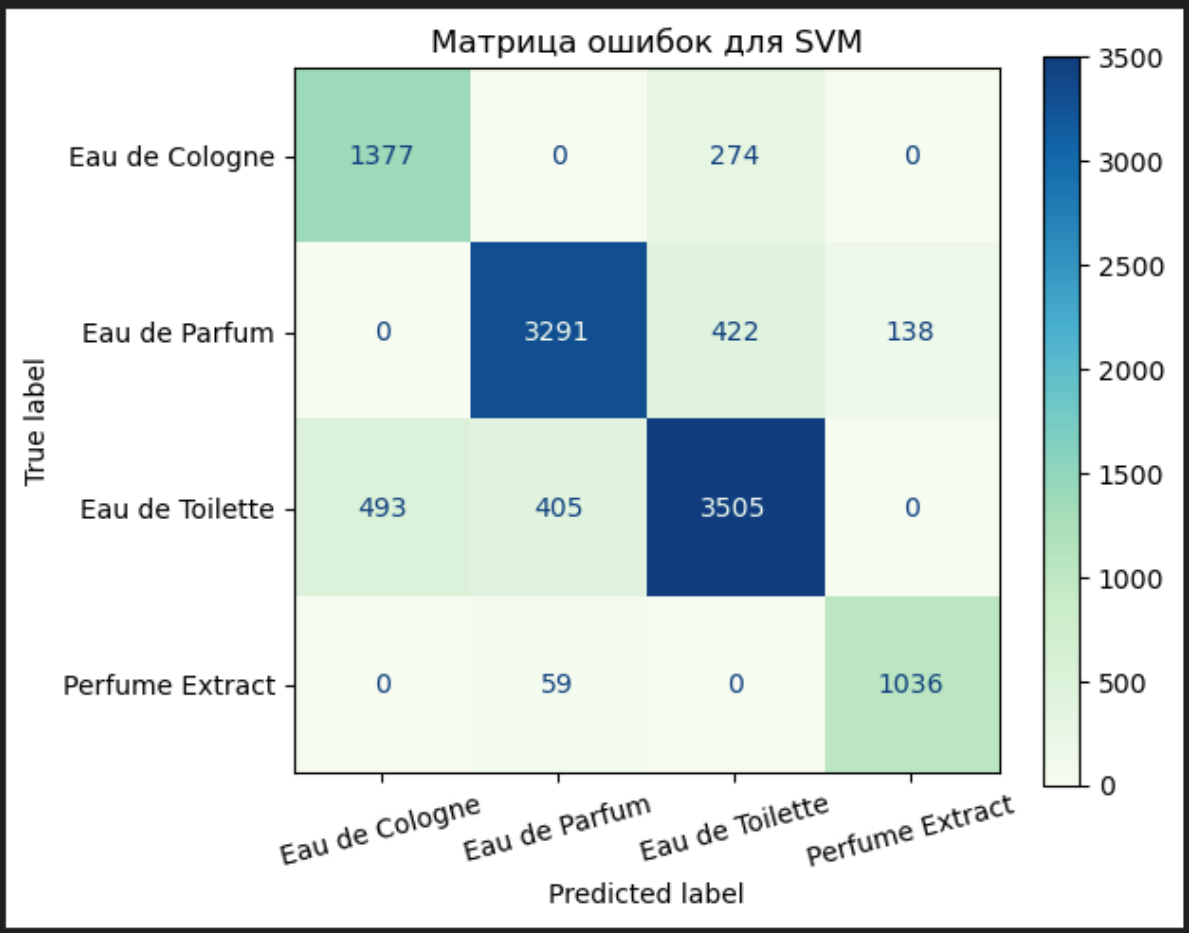


Рисунок 10 – Classification report и confusion matrix модели SVM

	precision	recall	f1-score	support
Eau de Cologne	0.76	0.80	0.78	1651
Eau de Parfum	0.87	0.86	0.87	3851
Eau de Toilette	0.83	0.81	0.82	4403
Perfume Extract	0.90	0.92	0.91	1095
accuracy			0.84	11000
macro avg	0.84	0.85	0.84	11000
weighted avg	0.84	0.84	0.84	11000

ROC-AUC Score (One-vs-Rest, Macro): 0.9636

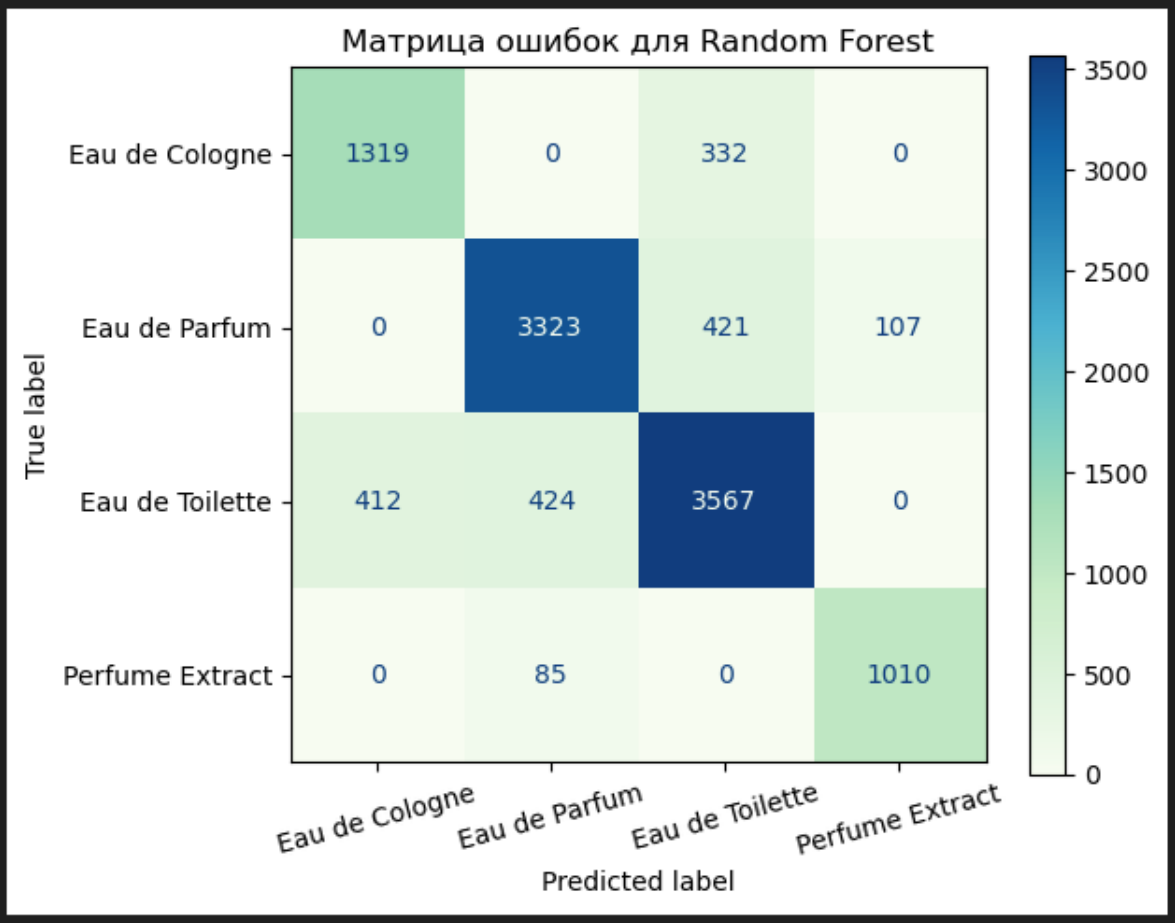


Рисунок 11 – Матрица ошибок модели Random Forest

Построен график важности признаков для случайного леса (рисунок 12).

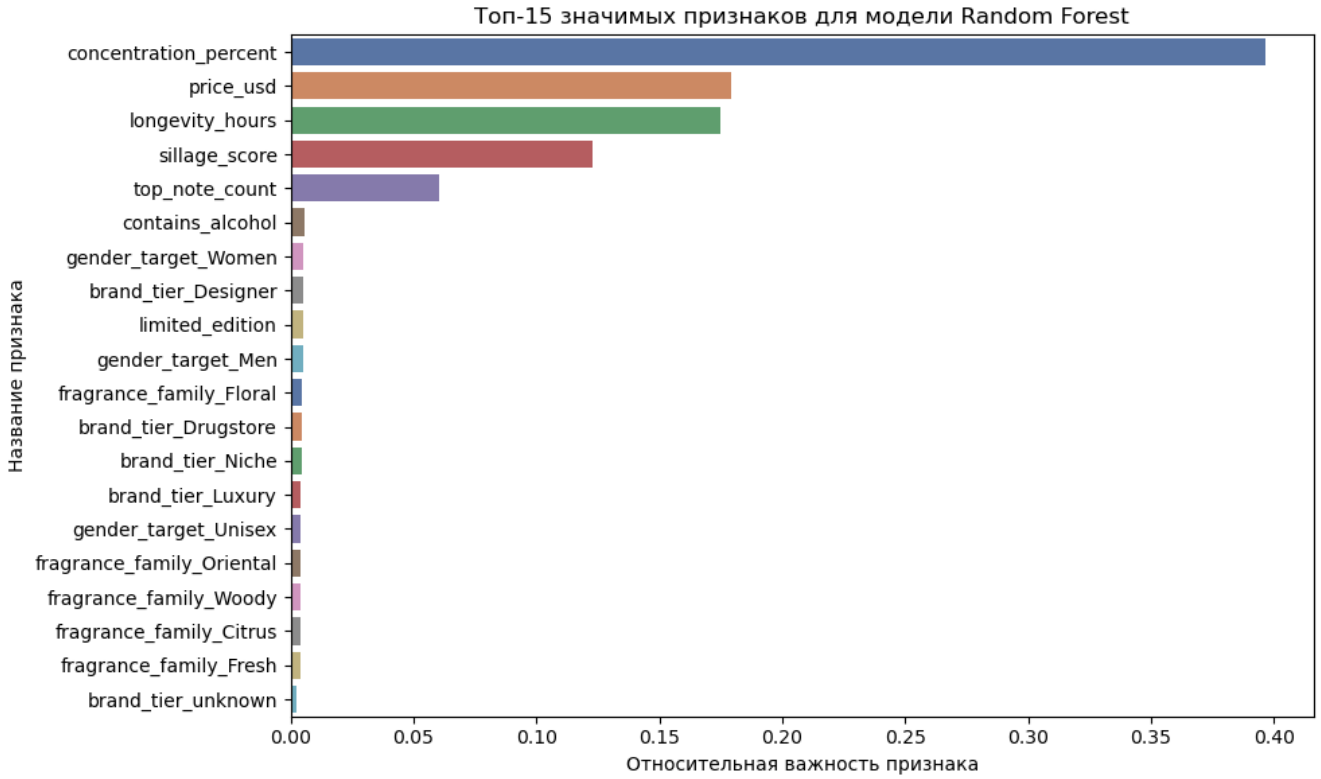


Рисунок 12 – Важность признаков для модели Random Forest

Вывод

В ходе выполнения лабораторной работы были получены и закреплены навыки предобработки данных и применения методов машинного обучения для задачи классификации.

Выбранный датасет Perfume Type Classification содержит 70 000 записей, 11 признаков, 4 таргетных класса и ярко выраженный дисбаланс. Для борьбы с дисбалансом был написан собственный скрипт генерации данных на основе SMOTE.

В работе обучены три модели: KNN, SVM и Random Forest. Для каждой выполнен подбор гиперпараметров с помощью GridSearchCV с 3-кратной кросс-валидацией.

Модели Random Forest и SVM classifier обе показали достаточно хорошие результаты с макро F1 = 0.96. Модель KNN показала худшее качество, что может объясняться большой чувствительностью к выбросу, шумам и масштабу.

Анализ важности признаков для случайного леса показал, что наибольший вклад в предсказание вносят признаки: concentration_percent, price_usd, longetivity_hours, sillage_score, top_note_count.

Приложение А

Исходный код программы

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from imblearn.base import BaseSampler
from imblearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    classification_report,
    confusion_matrix,
    roc_auc_score,
)
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder, OneHotEncoder,
StandardScaler
from sklearn.svm import SVC

# Загрузка и первичный анализ данных
dataset_path = "perfume_type_classification_dataset.csv"
df = pd.read_csv(dataset_path)
print(df.head())

# Справка по датасету:
# longevity_hours       - How long the scent lasts
# sillage_score         - Scent trail/projection strength (1-10)
# top_note_count          - Number of top notes (fewer in stronger
concentrations)
# price_usd             - Retail price
# concentration_percent - Fragrance oil concentration (%)
# fragrance_family      - Floral / Woody / Oriental / Fresh / Citrus
# brand_tier            - Luxury / Designer / Niche / Drugstore
# gender_target         - Men / Women / Unisex
# contains_alcohol      - 0/1 yes or no
# limited_edition       - 0/1 yes or no
# perfume_category       - Eau de Cologne / Eau de Toilette / Eau de
Parfum / Perfume Extract

# Наш датасет содержит 70k строк, 10 признаков и 1 целевую переменную
- perfume_category
print(df.describe())
```

```
print(df.info())

# Очистка и исследование признаков
print(f"Дубликатов найдено: {df.duplicated().sum()}")
df = df.drop_duplicates()
print("Пропущенные значения по колонкам:")
print(df.isnull().sum())

X = df.drop(columns="perfume_category")
y = df["perfume_category"]

# Визуализация распределения целевой переменной
sns.countplot(data=df, x="perfume_category")
plt.show()

# Кодирование целевой переменной и разбиение на выборки
target_encoder = LabelEncoder()
y = target_encoder.fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Разделение признаков на категориальные и числовые
cat_cols = [name for name in X.columns if X[name].dtype == object]
num_cols = [
    name
    for name in X.columns
    if X[name].dtype == float or df[name].dtype == int
]

# Заполнение пропусков
df.loc[:, num_cols] = df[num_cols].fillna(df[num_cols].median())
df.loc[:, cat_cols] = df[cat_cols].fillna("Unknown")

# Визуализация распределения отдельных признаков
fig, axes = plt.subplots(1, 3, figsize=(12, 3))
sns.kdeplot(data=df, x=df["contains_alcohol"], ax=axes[0], fill=True)
sns.kdeplot(data=df, x=df["price_usd"], ax=axes[1], fill=True)
sns.countplot(data=df, x=df["brand_tier"], ax=axes[2])
plt.tight_layout()
plt.show()

# Быстрая оценка важности признаков с помощью Random Forest
X_temp = pd.get_dummies(X, columns=cat_cols, drop_first=True)
rf_for_features = RandomForestClassifier(random_state=42)
rf_for_features.fit(X_temp, y)
importances = rf_for_features.feature_importances_
indices = np.argsort(importances)[::-1]
```

```
plt.figure(figsize=(8, 4))
plt.bar(range(10), importances[indices[:10]], align="center")
plt.xticks(
    range(10),  X_temp.columns[indices[:10]].to_list(),  rotation=45,
ha="right"
)
plt.tight_layout()
plt.show()

# Построение матрицы корреляции
plt.figure(figsize=(12, 10))
corr_df = X_temp.copy()
corr_df["target"] = y
sns.heatmap(corr_df.corr(), annot=True, cmap="coolwarm")
plt.show()

# Визуализация плотности и распределения для concentration_percent
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
sns.kdeplot(
    x=X_temp["concentration_percent"],
    hue=y,
    fill=True,
    alpha=0.3,
    palette="deep",
    ax=axes[0],
)
sns.violinplot(
    y=X_temp["concentration_percent"],     hue=y,     palette="deep",
ax=axes[1]
)
plt.show()

# Визуализация для longevity_hours
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
sns.kdeplot(
    x=X_temp["longevity_hours"],
    hue=y,
    fill=True,
    alpha=0.4,
    palette="deep",
    ax=axes[0],
)
sns.violinplot(y=X_temp["longevity_hours"],   hue=y,   palette="deep",
ax=axes[1])
plt.show()

# Комплексный анализ распределения price_usd
fig, axes = plt.subplots(1, 4, figsize=(20, 4))
sns.kdeplot(
```

```
    x=X_temp["price_usd"],      hue=y,      fill=True,      alpha=0.3,
palette="deep", ax=axes[0]
)
sns.violinplot(y=X_temp["price_usd"],      hue=y,      palette="deep",
ax=axes[1])
sns.scatterplot(x=X_temp["price_usd"],  y=y,  hue=y,  palette="deep",
ax=axes[2])
sns.histplot(x=X_temp["price_usd"],  kde=True,  ax=axes[3],  bins=40,
color="blue")
plt.show()

# Визуализация для sillage_score
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
sns.kdeplot(
    x=X_temp["sillage_score"],
    hue=y,
    fill=True,
    alpha=0.3,
    palette="deep",
    ax=axes[0],
)
sns.violinplot(y=X_temp["sillage_score"],    hue=y,    palette="deep",
ax=axes[1])
plt.show()


# Реализация кастомной функции SMOTE для ребалансировки классов
from imblearn import FunctionSampler


def custom_smote_func(
    X,   y,   target_class,   oversampling_rate=1.0,   k_neighbors=5,
cat_indices=None
):
    """Функция   для   генерации   синтетических   данных   (кастомная
реализация SMOTE)"""
    if cat_indices is None:
        cat_indices = []
    X_arr = np.asarray(X, dtype=float)
    y_arr = np.asarray(y)

    # Берем выборку класса, который будем балансировать
    minority_samples = X_arr[y_arr == target_class]
    n_samples, n_features = minority_samples.shape
    if n_samples <= k_neighbors:
        raise ValueError(
            f"Количество объектов класса {target_class} ({n_samples})
слишком мало для k_neighbors={k_neighbors}"
        )
```

```
    n_synthetic = int(n_samples * oversampling_rate)
    synthetic_samples = np.empty((n_synthetic, n_features))

    for i in range(n_synthetic):
        # Генерируем случайный индекс для выбора базового объекта
        base_idx = np.random.randint(0, n_samples)
        base_sample = minority_samples[base_idx]

        # Считаем расстояния и находим ближайших соседей
        distances = np.linalg.norm(minority_samples - base_sample,
axis=1)
        nn_indices = np.argsort(distances)[1 : k_neighbors + 1]

        # Выбираем случайного соседа
        chosen_neighbor_idx = np.random.choice(nn_indices)
        neighbor_sample = minority_samples[chosen_neighbor_idx]

        # Вычисляем случайное смещение
        gap = np.random.rand()
        synthetic_specimen = base_sample + gap * (neighbor_sample -
base_sample)

        # Наследуем категории от базового объекта для избежания проблем
с OHE
        for idx in cat_indices:
            synthetic_specimen[idx] = base_sample[idx]
        synthetic_samples[i] = synthetic_specimen

    X_balanced = np.vstack((X_arr, synthetic_samples))
    y_synthetic = np.full(n_synthetic, target_class)
    y_balanced = np.concatenate((y_arr, y_synthetic))

    return X_balanced, y_balanced


print(df["perfume_category"].value_counts())
print(list(target_encoder.classes_).index("Eau de Cologne"))
print(list(target_encoder.classes_).index("Perfume Extract"))

# Пайплайны предобработки
categorical_pipe = Pipeline(
    steps=[
        ("cat_imputer",            SimpleImputer(strategy="constant",
fill_value="unknown")),
        (
            "one_hot_encoder",
            OneHotEncoder(handle_unknown="ignore",
sparse_output=False),
```

```
        ),
    ]
)

numerical_pipe = Pipeline(
    steps=[
        ("num_imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("cat_tf", categorical_pipe, cat_cols),
        ("num_tf", numerical_pipe, num_cols),
    ]
)

preprocessor.fit(X_train)
num_cat_features_after_ohe = (
    preprocessor.named_transformers_["cat_tf"]["one_hot_encoder"]
    .get_feature_names_out()
    .shape[0]
)
cat_indices = list(range(num_cat_features_after_ohe))

# Словарь моделей и сеток параметров для GridSearchCV
model_tuning_configs = {
    "KNN": {
        "model": KNeighborsClassifier(),
        "params": {
            "classifier__n_neighbors":,
            "classifier__weights": ["uniform", "distance"],
        },
    },
    "SVM": {
        "model": SVC(probability=True, random_state=42),
        "params":      {"classifier__C":      [0.1,      1,      10],
"classifier__kernel": ["rbf"]},
    },
    "Random Forest": {
        "model": RandomForestClassifier(random_state=42),
        "params": {
            "classifier__n_estimators":,
            "classifier__max_depth": [10, 20, None],
        },
    },
}
```

```
best_models = {}

# Обучение и тюнинг моделей
for model_name, config in model_tuning_configs.items():
    print(f"Запуск GridSearchCV для {model_name}...")

    # Оборачиваем функцию в FunctionSampler для каждого класса отдельно
    smote_class_0 = FunctionSampler(
        func=custom_smote_func,
        kw_args={
            "target_class": 0,
            "oversampling_rate": 1.0,
            "k_neighbors": 3,
            "cat_indices": cat_indices,
        },
    )
    smote_class_3 = FunctionSampler(
        func=custom_smote_func,
        kw_args={
            "target_class": 3,
            "oversampling_rate": 3.0,
            "k_neighbors": 3,
            "cat_indices": cat_indices,
        },
    )

    pipeline = Pipeline(
        steps=[
            ("preprocessor", preprocessor),
            ("smote_class_0", smote_class_0),
            ("smote_class_3", smote_class_3),
            ("classifier", config["model"]),
        ]
    )
    grid_search = GridSearchCV(
        estimator=pipeline,
        param_grid=config["params"],
        cv=3,
        scoring="f1_macro",
        n_jobs=-1,
        verbose=1,
    )
    grid_search.fit(X_train, y_train)

    print(f"Лучшие параметры для {model_name}:
{grid_search.best_params_}")
    print(f"Лучший F1-Macro на кросс-валидации:
{grid_search.best_score_:.4f}\n")
    best_models[model_name] = grid_search.best_estimator_
```